TP 3 – Composants & cycle de vie

Catalogue de livres (refonte en composants)

Pr. Ahmed Bentajer

Objectifs

- Découper une application en composants monofichiers (SFC)
- Faire communiquer parent et enfants avec `props` et `$emit`
- Valider des props (type, requis, valeur par défaut)
- Utiliser un slot nommé et le hook `mounted`

Mise en place

À partir de ce TP, le projet passe à Vite (l'outil recommandé aujourd'hui pour démarrer un projet Vue, en remplacement de Vue CLI). Le dossier `starter/` est un projet Vite complet :

```
cd starter
npm install
npm run dev
```

La structure attend trois composants à compléter dans `src/components/` : `AddBookForm.vue`, `BookList.vue` et `BookCard.vue`. `App.vue` reste le seul endroit où vit la liste des livres (`books`) – c'est lui qui « possède » l'état, les enfants ne font que l'afficher ou demander une modification.

1 Props et affichage

Dans `BookList.vue`, déclarez une props `books` (tableau requis), bouclez dessus avec `v-for`/`:key` et instanciez `<BookCard :book="book" :key="book.id" />` pour chacun. Dans `App.vue`, passez `:books="books"` à `<BookList>`.

2 Valider la props book

Dans `BookCard.vue`, remplacez la déclaration vide de `book` par un objet complet : type `Object`, `required: true`, avec un `validator` qui vérifie que `book.title` existe et n'est pas vide.

3 Ne pas muter une props

Le bouton « Favori » de `BookCard.vue` contient une ligne fournie `@click="book.isFav = !book.isFav"`.

Piège classique

`book` est une **props** : elle appartient au parent. Vue autorise cette écriture sans planter, mais le flux de données à sens unique du framework est cassé – si `App.vue` recalculait un jour `books` depuis une autre source, la modification locale serait silencieusement écrasée. Le cours est explicite sur ce point (diapo « props : flux à sens unique ») : quand un enfant a besoin de faire évoluer une donnée reçue en props, il doit le demander au parent via `$emit`, jamais modifier la props directement.

4 Slot nommé

Ajoutez à `BookCard.vue` un slot nommé `actions`, placé après le badge favori. Depuis `BookList.vue` (ou directement `App.vue` si vous préférez), injectez-y un petit texte ou bouton de votre choix pour vérifier que le contenu du parent s'affiche bien à l'endroit du slot.

5 Chargement initial (mounted)

Dans `App.vue`, complétez le hook `mounted()` pour simuler un chargement réseau : à l'intérieur d'un `setTimeout(() => { ... }, 600)`, affectez `initialBooks` à `this.books` puis passez `loading` à `false`. Utilisez `v-show="loading"` sur le message de chargement (rappel : `v-show`, pas `v-if`, puisque l'élément change souvent d'état ici – exactement le cas d'usage vu en cours).

Bonus – ref template

Dans `AddBookForm.vue`, ajoutez `ref="titreInput"` sur le premier champ, puis dans un hook `mounted()` appelez `this.$refs.titreInput.focus()` pour que le curseur se place automatiquement dans le champ titre à l'ouverture du formulaire.

Ce TP se poursuit au TP4 : vous ajouterez la navigation (Vue Router) et convertirez un composant en Composition API.